МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных технологий

Кафедра
управления и информатики
в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ

ЛАБОРАТОРНОЙ РАБОТЫ ПО ДИСЦИПЛИНЕ

«Прикладное программирование**»**

по теме «ООП в Python. Обработка исключительных ситуаций. Форматы XML и JSON»

СТУДЕНТА *2* КУРСА бакалавриата ГРУППЫ ИДБ-25-12

(уровень профессионального образования)

Яушева Сафия Рустамовна

Направление: 09.03.03 Прикладная информатика

Отчет сдан «_____» _____2026г.

Оценка _____

Преподаватель Гетманов С. В. _____

(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

Оглавление

1. Постановка задачи 3

2. Описание предметной области 4

3. Проектирование классов UML 5

4. Реализация программы 8

4.1. Прототипы пользовательского интерфейса 10

5. Обработка исключительных ситуаций 12

5.1. Проверка вводимых данных 14

6. Хранение данных в JSON и XML 14

7. Проверка работы программы 16

8. Заключение 18

Приложение. Структура проекта 20

1. Постановка задачи

Для предметной области «Сервис доставки» (вариант 26) необходимо спроектировать диаграмму классов в draw.io и написать по ней код на Python в объектно-ориентированном стиле; обработать встроенные и собственные исключения так, чтобы ошибки пользователя не завершали программу, и актуализировать диаграмму; спроектировать структуру хранения данных в форматах XML и JSON и реализовать чтение из файла и запись в файл. Работа выполнялась в системе контроля версий Git, исходный код размещён в публичном репозитории GitHub (https://github.com/schnchladp/stankin-lab).

2. Описание предметной области

Сервис доставки принимает от клиентов заказы на перевозку посылок. Клиент оформляет заказ: указывает описание, вес и объявленную стоимость посылки, признак хрупкости, адреса отправления и получения, расстояние. Сотрудник сервиса назначает на заказ свободного курьера и транспорт (автомобиль, самокат или велосипед), в результате создаётся доставка, а курьер становится занятым. После завершения доставки курьер снова свободен, а клиент может оплатить заказ и оставить отзыв с оценкой от 1 до 5.

Стоимость рассчитывается по тарифу: максимум из минимальной цены и суммы «расстояние*цена за километр + вес*цена за килограмм»; для экспресс-доставки и хрупких посылок применяются повышающие коэффициенты. Заказ и доставка проходят через статусы «Создан», «В процессе доставки», «Доставлен» и «Отменён», причём допустимы только определённые переходы.

Модель учебная: не учитываются построение маршрутов, геокодирование адресов, реальные платежи и персональные данные. Все данные демонстрационного набора вымышлены.

3. Проектирование классов UML

Диаграмма включает одиннадцать классов предметной области (Person, Customer, Courier, Address, Parcel, Order, Transport, Delivery, Tariff, Payment, Review) и перечисление Status, три класса управления и хранения (DeliveryService, DataStorage, DeliveryApp) и восемь классов исключений. Назначение классов приведено в таблице 1.

Таблица 1. Назначение классов

Класс	**Назначение**
Person	абстрактный человек: имя, телефон, проверка и нормализация контактных данных, абстрактный метод describe()
Customer, Courier	клиент и курьер (наследники Person); у курьера есть удостоверение и признак доступности
Address	адрес отправления или получения (dataclass)
Parcel	посылка: описание, вес, объявленная стоимость, признак хрупкости
Order	заказ: клиент, посылка, два адреса, расстояние, статус
Transport	транспорт курьера: тип (car, scooter, bike) и госномер (только у автомобиля, формат A123BC77)
Delivery	доставка: заказ, курьер, транспорт, статус
Tariff	тариф и формула расчёта стоимости
Payment, Review	оплата заказа и отзыв клиента о доставке
Status	перечисление статусов и допустимые переходы между ними
DeliveryService	хранит все коллекции, выполняет операции CRUD и бизнес-правила
DataStorage	запись и чтение данных в форматах JSON и XML
DeliveryApp	консольное меню пользователя

Основные связи: Customer и Courier наследуются от абстрактного класса Person, Order связан с клиентом ассоциацией и включает посылку (композиция) и два адреса, Delivery связана с заказом и курьером и включает транспорт, Payment ссылается на заказ, Review – на клиента и доставку, Tariff используется для расчёта стоимости заказа. Класс DeliveryService агрегирует коллекции всех основных сущностей, а DataStorage и DeliveryApp работают через него.

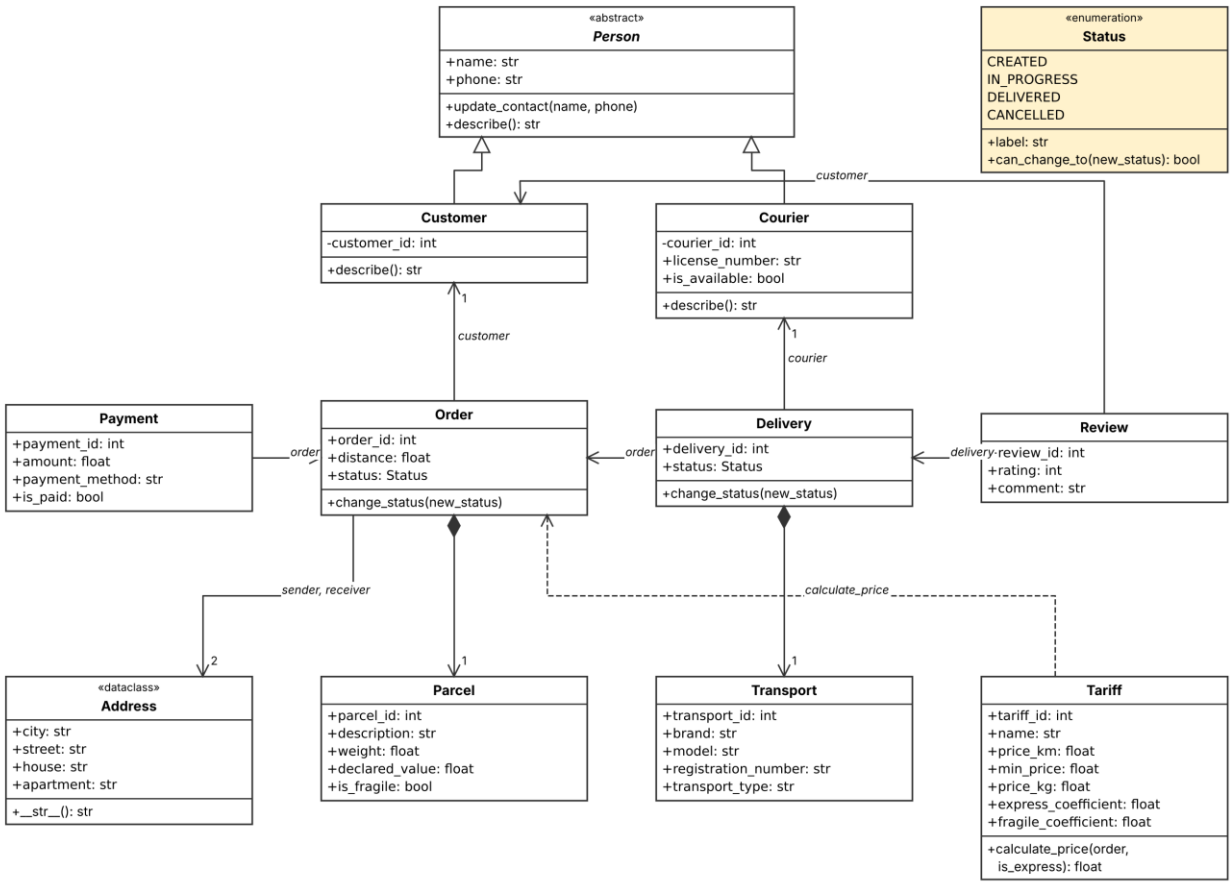


Рис. 1. Диаграмма классов предметной области (до добавления исключений)

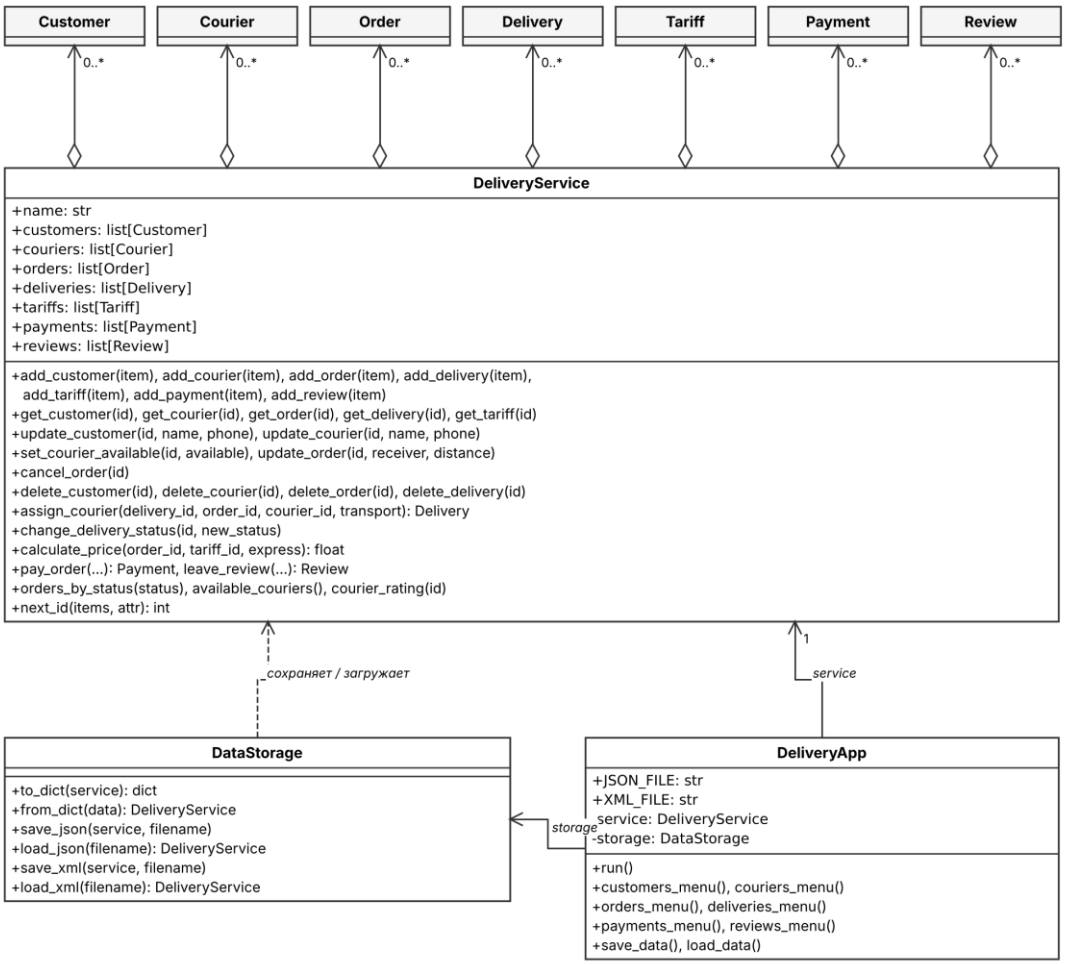


Рис. 2. Диаграмма классов: слой управления, хранения и интерфейса

4. Реализация программы

Программа написана на Python 3.10+ с использованием только стандартных модулей: dataclasses, abc, enum, re, json, xml.etree.ElementTree, pathlib, contextlib и typing. Графического интерфейса нет, работа ведётся через консольное меню (запуск: python main.py). Код разделён на модули пакета delivery: models.py (сущности), service.py (DeliveryService), storage.py (JSON и XML), exceptions.py (исключения) и app.py (меню).

Принципы ООП: инкапсуляция – данные проверяются в конструкторах и в методе update_contact() с помощью функций normalize_name, normalize_phone и normalize_plate, выдавая InvalidDataError; наследование – Customer и Courier наследуют абстрактный Person; полиморфизм – метод describe() по-разному реализован в наследниках; перечисление Status хранит допустимые переходы статусов (метод can_change_to). Во всех функциях указаны аннотации типов, код оформлен по PEP 8 (строки до 79 символов, docstring и комментарии), обобщённые вспомогательные методы используют TypeVar.

Операции CRUD для основных сущностей приведены в таблице 2. Идентификаторы в меню присваиваются автоматически (максимальный + 1), поэтому пользователь не вводит их вручную при создании.

Таблица 2. Операции CRUD в классе DeliveryService

Сущность	**Создание**	**Поиск**	**Изменение**	**Удаление**
Клиент	add_customer	get_customer	update_customer	delete_customer
Курьер	add_courier	get_courier, available_couriers	update_courier, set_courier_available	delete_courier
Заказ	add_order	get_order, orders_by_status	update_order, cancel_order	delete_order
Доставка	assign_courier	get_delivery	change_delivery_status	delete_delivery
Тариф	add_tariff	get_tariff, calculate_price	–	–

Оплата	pay_order	список оплат	–	–
Отзыв	leave_review	courier_rating	–	–

Бизнес-правила: курьера можно назначить только на новый заказ и только если он свободен; при назначении заказ переходит в статус «В процессе доставки», курьер становится занятым, а при завершении или отмене доставки статусы заказа и доставки меняются, и курьер освобождается; заказ можно изменить только до назначения курьера; удалять клиента с заказами, курьера с доставками, оплаченный заказ или активную доставку нельзя; отзыв может оставить только владелец заказа и только после доставки.

4.1. Прототипы пользовательского интерфейса

Макет 1. Главное меню

```
 ------------------------------------------------------------
| СЛУЖБА КУРЬЕРСКОЙ ДОСТАВКИ                                 |
 ------------------------------------------------------------
| 1. Клиенты                                                 |
| 2. Курьеры                                                 |
| 3. Заказы                                                  |
| 4. Доставки                                                |
| 5. Оплата и тарифы                                         |
| 6. Отзывы                                                  |
| 7. Сохранить данные (JSON и XML)                           |
| 8. Загрузить данные                                        |
| 0. Выход                                                   |
 ------------------------------------------------------------
| Выберите пункт: _                                          |
 ------------------------------------------------------------
```

Макет 2. Карточка заказа

```
 ---------------------------------------------------------------------------
| ЗАКАЗ №1                                                                  |
 ---------------------------------------------------------------------------
| Статус:       Доставлен                                                   |
| Клиент:       Алина                                                       |
| Посылка:      Ваза, 3.5 кг, хрупкая                                       |
| Откуда:       Москва, ул. Тверская, д. 10, кв. 25                         |
| Куда:         Москва, ул. Арбат, д. 15                                    |
| Расстояние:   5.0 км                                                      |
 ---------------------------------------------------------------------------
| [Изменить адрес] [Отменить заказ] [Назначить курьера]                     |
 ---------------------------------------------------------------------------
```

Макет 3. Меню «Заказы»

```
 ------------------------------------------------------------
| ЗАКАЗЫ                                                     |
 ------------------------------------------------------------
| 1. Список                                                  |
| 2. Создать                                                 |
| 3. Найти по id                                             |
| 4. Изменить адрес получателя                               |
| 5. Отменить заказ                                          |
| 6. Заказы по статусу                                       |
| 7. Удалить                                                 |
| 0. Назад                                                   |
```

```
 ------------------------------------------------------------
| Выберите пункт: _                                          |
 ------------------------------------------------------------
```

Макеты показывают структуру экранов. Реализованная программа работает в консольном режиме, макет 2 соответствует выводу пункта «Найти по id».

5. Обработка исключительных ситуаций

Созданы собственные исключения с общим предком DeliveryError (таблица 3, рис. 3). Проверяются пустые обязательные поля, формат имени, телефона и госномера, положительность веса, расстояния и суммы, диапазон оценки, допустимость статуса и перехода между статусами, существование связанных объектов и уникальность идентификаторов.

Таблица 3. Собственные исключения

Исключение	**Когда возникает**
DeliveryError	базовый класс ошибок приложения
InvalidDataError	пустое имя, неверный телефон, вес или расстояние не больше нуля, оценка вне диапазона 1–5, повторная оплата
NotFoundError	объект с таким id не найден
DuplicateIdError	объект с таким id уже существует
DependencyError	удаление объекта, на который есть ссылки (клиент с заказами, курьер с доставками, активная доставка)
InvalidStatusError	недопустимая смена статуса (например, отмена доставленного заказа)
CourierUnavailableError	назначение занятого курьера
StorageError	ошибка чтения или записи файла JSON/XML

Встроенные исключения обрабатываются так: ValueError – при вводе чисел и ответов «да/нет» в меню; FileNotFoundError и OSError – при недоступном файле; json.JSONDecodeError и xml.etree.ElementTree.ParseError – при повреждённом JSON или XML; KeyError, TypeError, AttributeError и ValueError – при файле неверной структуры. Все файловые ошибки преобразуются в StorageError с понятным сообщением (контекстный менеджер _translate_errors в storage.py).

Каждое действие меню выполняется внутри _safe_call(), который перехватывает DeliveryError, ValueError и любое непредвиденное исключение Exception, печатает сообщение и возвращает пользователя в меню. Окончание

ввода (EOFError) и прерывание (Ctrl+C) обрабатываются в run() с корректным выходом.

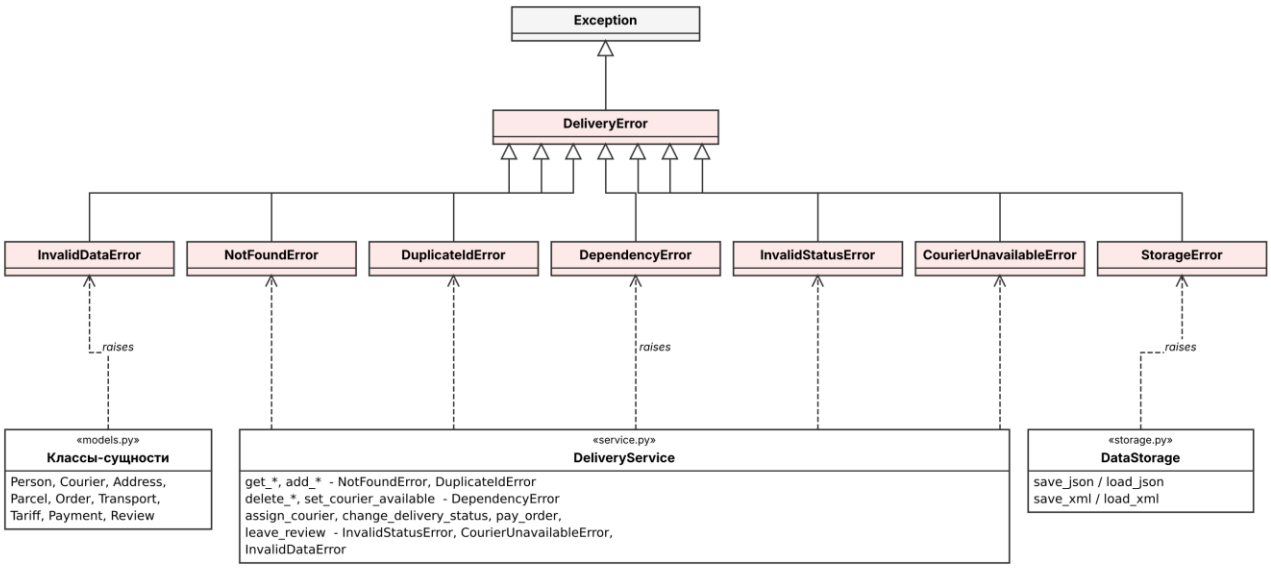


Рис. 3. Актуализированная диаграмма: иерархия исключений и места их возникновения

5.1. Проверка вводимых данных

Данные проверяются в момент создания объекта или изменения полей (модуль models.py), поэтому некорректное значение не попадает ни в память, ни в файл: при загрузке из JSON и XML объекты создаются теми же конструкторами и проходят те же проверки. Имя, телефон и госномер проверяются регулярными выражениями NAME_PATTERN, PHONE_PATTERN и PLATE_PATTERN (модуль re). Правила приведены в таблице 4.

Таблица 4. Правила проверки данных

Поле	**Правило проверки**
Имя клиента и курьера	только буквы; слова разделяются одним пробелом или дефисом (Мария-Анна Ли); пробелы по краям и двойные пробелы убираются; цифры и знаки недопустимы
Телефон	мобильный номер в формате +79XXXXXXXXX или 89XXXXXXXXX; пробелы, дефисы и скобки допускаются; хранится в единой записи +79XXXXXXXXX; уникален среди клиентов и среди курьеров
Госномер автомобиля	формат А123ВС77 или А123ВС777: буква, три цифры, две буквы, код региона из двух-трёх цифр; буквы только русские АВЕКМНОРСТУХ, строчные приводятся к заглавным; цифры 000 и регион 00 недопустимы; при вводе программа показывает нужный формат
Госномер самоката и велосипеда	номера нет (в России они не регистрируются), поле должно быть пустым
Вес посылки, расстояние, сумма оплаты	строго больше нуля
Объявленная стоимость	не меньше нуля
Оценка в отзыве	целое число от 1 до 5
Описание посылки, номер удостоверения, город, улица, дом	не должны быть пустыми
Тариф	цены не отрицательные, коэффициенты не меньше 1

6. Хранение данных в JSON и XML

В обоих форматах данные разбиты на семь коллекций: customers, couriers, tariffs, orders, deliveries, payments, reviews. Связи между объектами заданы

числовыми идентификаторами; посылка и адреса вложены в заказ, транспорт – в доставку. Статусы хранятся строками (created, in_progress, delivered, cancelled).

Фрагмент JSON (data/data.json):

```
{
    "name": "Служба курьерской доставки",
    "customers": [
        {"id": 1, "name": "Алина", "phone": "+79000000001"}
    ],
    "orders": [
        {
            "id": 1,
            "customer_id": 1,
            "parcel_id": 1,
            "description": "Ваза",
            "weight": 3.5,
            "declared_value": 2500.0,
            "fragile": true,
            "distance": 5.0,
            "status": "delivered",
            "sender": {"city": "Москва", "street": "Тверская",
                       "house": "10", "apartment": "25"},
            "receiver": {"city": "Москва", "street": "Арбат",
                         "house": "15", "apartment": ""}
        }
    ],
    ...
}
```

Фрагмент XML (data/data.xml):

```
<delivery_service name="Служба курьерской доставки">
    <customers>
        <customer>
            <id>1</id>
            <name>Алина</name>
            <phone>+79000000001</phone>
        </customer>
    </customers>
    <deliveries>
        <delivery>
            <id>1</id>
            <order_id>1</order_id>
            <courier_id>1</courier_id>
            <transport>
                <id>1</id>
                <registration_number>A123BC77</registration_number>
                <transport_type>car</transport_type>
                ...
            </transport>
            <status>delivered</status>
        </delivery>
    </deliveries>
    ...
</delivery_service>
```

Запись и чтение реализованы классом DataStorage: save_json/load_json (модуль json) и save_xml/load_xml (модуль xml.etree.ElementTree, дерево форматируется отступами). Объекты сначала преобразуются в словарь (to_dict),

затем записываются в выбранном формате; при загрузке значения XML (текст) преобразуются в числа и логические значения функциями _to_int, _to_float и _to_bool, после чего объекты создаются заново с теми же проверками. Таким образом реализованы и сериализация, и десериализация.

7. Проверка работы программы

Таблица 5. Выполненные проверки

Проверка	**Ожидаемый результат**
Создание данных	Создаются клиенты, курьеры, заказы с посылками и адресами.
Назначение курьера	Курьер становится занятым, заказ переходит в статус «В процессе доставки».
Расчёт стоимости	Заказ №1 (5 км, 3,5 кг, хрупкий, экспресс): (5·100 + 3,5·20)·1,5·1,2 = 1026 руб.
Завершение доставки	Доставка и заказ получают статус «Доставлен», курьер снова свободен; доступны оплата и отзыв.
Проверка полей	Имя с цифрой или пустое, телефон «12-34» и не мобильный (+7495…), неверный госномер, номер у велосипеда, отрицательный вес, оценка 7 отклоняются InvalidDataError.
Проверка уникальности	Повторный id или повторный телефон вызывает DuplicateIdError.
Поиск	Несуществующий заказ или клиент вызывает NotFoundError.
Занятый курьер	Назначение занятого курьера вызывает CourierUnavailableError.
Статусы	Повторное завершение, отмена доставленного заказа, повторное назначение вызывают InvalidStatusError.
Целостность данных	Удаление клиента с заказами и заказа с доставкой вызывает DependencyError.
JSON и XML	Данные записываются и читаются обратно, результат совпадает с исходным.
Повреждённые файлы	Неверный JSON, XML, структура, дубликаты и отсутствие файла дают StorageError с причиной.
Консольное меню	Неверный пункт, нечисловой ввод и ошибки бизнес-логики не завершают программу.

Результат работы сценария demo.py (все ожидаемые ошибки обработаны, программа дошла до конца):

```
--- Нормальная работа ---
[OK] заказы созданы:
     №1 | Алина | Ваза | Создан
     №2 | Игорь | Документы | Создан
[OK] курьер назначен: ID: 1 | Заказ №1 | Курьер: Аделина | В процессе доставки
[OK] свободные курьеры: ['Пётр']
```

```
[OK] стоимость экспресс-доставки заказа №1: 1026.0 руб.
[OK] оплата принята: 1026.0 руб.
[OK] доставка завершена: ID: 1 | Заказ №1 | Курьер: Аделина | Доставлен
[OK] рейтинг курьера №1: 5.0
--- Обработка ошибок ---
[ОШИБКА] пустое имя: InvalidDataError: Имя не должно быть пустым
[ОШИБКА] имя с цифрами: InvalidDataError: Имя может содержать только буквы, пробелы и дефис: Олег2
[ОШИБКА] неверный телефон: InvalidDataError: Некорректный телефон: 12-34 (нужен формат +79XXXXXXXXX
или 89XXXXXXXXX)
[ОШИБКА] телефон не мобильный (+7495...): InvalidDataError: Некорректный телефон: +74951234567
(нужен формат +79XXXXXXXXX или 89XXXXXXXXX)
[ОШИБКА] повторный телефон: DuplicateIdError: Клиент с телефоном +79000000001 уже есть
[ОШИБКА] неверный госномер: InvalidDataError: Некорректный госномер: 1Фы45 (пример: А123ВС77, буквы
русские: АВЕКМНОРСТУХ)
[ОШИБКА] номер у велосипеда: InvalidDataError: Госномер есть только у автомобиля (car)
[ОШИБКА] отрицательный вес: InvalidDataError: Вес посылки должен быть больше нуля
[ОШИБКА] повторный id: DuplicateIdError: Клиент с id=1 уже есть
[ОШИБКА] нет такого заказа: NotFoundError: Заказ с id=99 не найден(а)
[ОШИБКА] курьер занят: CourierUnavailableError: Курьер Пётр сейчас занят
[ОШИБКА] заказ уже в доставке: InvalidStatusError: Курьера можно назначить только на новый заказ, а
сейчас: В процессе доставки
[ОШИБКА] повторная доставка завершена: InvalidStatusError: Нельзя перейти из статуса «Доставлен» в
«Доставлен»
[ОШИБКА] отмена доставленного заказа: InvalidStatusError: Нельзя перейти из статуса «Доставлен» в
«Отменён»
[ОШИБКА] повторная оплата: InvalidDataError: Этот заказ уже оплачен
[ОШИБКА] чужой отзыв: InvalidDataError: Это не доставка данного клиента
[ОШИБКА] оценка вне диапазона: InvalidDataError: Оценка должна быть от 1 до 5
[ОШИБКА] удаление клиента с заказами: DependencyError: У клиента есть заказы, удалять нельзя
[ОШИБКА] удаление заказа с доставкой: DependencyError: На заказ назначена доставка
--- JSON и XML ---
JSON: записано, прочитано, данные совпадают = True
XML:  записано, прочитано, данные совпадают = True
--- Повреждённые файлы ---
[ОШИБКА] загрузка bad.json: StorageError: Файл data/demo/bad.json - некорректный JSON: Expecting
property name enclosed in double quotes: line 1 column 2 (char 1)
[ОШИБКА] загрузка bad.xml: StorageError: Файл data/demo/bad.xml - некорректный XML: no element
found: line 1, column 9
[ОШИБКА] загрузка wrong.json: StorageError: Файл data/demo/wrong.json имеет неверную структуру
(ValueError: invalid literal for int() with base 10: 'x')
[ОШИБКА] загрузка dup.json: StorageError: В файле data/demo/dup.json некорректные данные: Клиент с
id=1 уже есть
[ОШИБКА] файла нет: StorageError: Файл data/demo/none.json не найден
Демонстрация завершена, программа дошла до конца.
```

Сеанс в консольном меню: имя с цифрой, телефон не в формате +79… / 89…, повторный телефон, неверный госномер, оформление заказа, отмена загрузки данных и несуществующий пункт меню. После каждой ошибки программа возвращается в меню.

```
Выберите пункт: 2
Имя: Алина
Телефон: +7 900 000-00-01
Клиент добавлен, id=1
Выберите пункт: 2
Имя: Игорь7
Телефон: +79000000002
Ошибка: Имя может содержать только буквы, пробелы и дефис: Игорь7
Выберите пункт: 2
Имя: Игорь
Телефон: +74951234567
Ошибка: Некорректный телефон: +74951234567 (нужен формат +79XXXXXXXXX или 89XXXXXXXXX)
Выберите пункт: 2
Имя: Игорь
```

```
Телефон: 8 (900) 000-00-01
Ошибка: Клиент с телефоном +79000000001 уже есть
Выберите пункт: 0
Выберите пункт: 2
Выберите пункт: 2
Имя: Аделина
Телефон: +79111111111
Номер удостоверения: 1234567890
Курьер добавлен, id=1
...
Выберите пункт: 2
Клиент №1: Алина, +79000000001
id клиента: 1
Описание посылки: Ваза
Вес (кг): 3,5
Объявленная стоимость (руб): 2500
Посылка хрупкая (да/нет): да
...
Расстояние (км): 5
Заказ создан, id=1
Выберите пункт: 0
Выберите пункт: 4
Выберите пункт: 2
id заказа: 1
id курьера: 1
1. car
2. scooter
3. bike
Тип транспорта: 1
Госномер в формате А123ВС77 (русские буквы АВЕКМНОРСТУХ): 1Фы45
Ошибка: Некорректный госномер: 1Фы45 (пример: А123ВС77, буквы русские: АВЕКМНОРСТУХ)
Выберите пункт: 2
id заказа: 1
id курьера: 1
1. car
2. scooter
3. bike
Тип транспорта: 1
Госномер в формате А123ВС77 (русские буквы АВЕКМНОРСТУХ): А123ВС77
Курьер назначен, доставка id=1
Выберите пункт: 0
Выберите пункт: 8
Текущие данные в программе будут заменены. Продолжить (да/нет): нет
Загрузка отменена
Выберите пункт: 9
Нет такого пункта меню
Выберите пункт: 0
До свидания!
```

8. Заключение

В ходе лабораторной работы спроектирована объектная модель сервиса доставки и реализована программа на Python с консольным меню. Модель включает одиннадцать классов предметной области, перечисление статусов, классы управления и хранения. Реализованы операции CRUD, правила назначения курьера и смены статусов, расчёт стоимости, оплата и отзывы. Данные проверяются при вводе: имя (только буквы, пробелы и дефис), мобильный телефон (+79… или 89…), госномер автомобиля РФ. Созданы

собственные исключения, благодаря которым ошибки пользователя и повреждённые файлы не приводят к аварийному завершению. Разработаны сохранение и загрузка данных в JSON и XML, подготовлены демонстрационные файлы и диаграммы классов UML. Поставленные задачи лабораторной работы выполнены.

Приложение. Структура проекта

main.py – запуск консольного приложения

demo.py – демонстрационный сценарий: обычная работа и все виды ошибок

delivery/models.py – классы предметной области

delivery/service.py – DeliveryService: операции CRUD и бизнес-правила

delivery/storage.py – DataStorage: запись и чтение JSON и XML

delivery/exceptions.py – собственные исключения

delivery/app.py – консольное меню

data/data.json, data/data.xml – демонстрационные наборы данных

docs/ – диаграммы классов (.drawio и .png)

README.md – описание проекта и инструкция по запуску

.gitignore – исключения для Git

Ссылка на репозиторий GitHub: https://github.com/schnchladp/stankin-lab/tree/main/lab1